

NeuroTrace

An observability layer for AI agent execution — capture every LLM call, tool call, and failure an agent makes, then inspect the run instead of re-running it blind.

Version

0.1.0 (released)

Status

Complete · 7 of 7 days

Date

21 July 2026

Owner

Dheeraj Pranav

Language

Python ≥ 3.10

Repository

github.com/DheerajPranav/neurotrace

— Contents

- 1. **Executive Summary** — what this is, in one page
- 2. **The Problem We Are Solving** — the business cost of opaque agents
- 3. **The Product** — what NeuroTrace does and who it serves
- 4. **Architecture** — layers, data flow, sequence, data model
- 5. **Component Walkthrough** — every file, its job, its business value
- 6. **Key Engineering Decisions** — the trade-offs and why they were made
- 7. **Capabilities** — what works today, verified
- 8. **Competitive Advantages** — why this instead of the alternatives
- 9. **Limitations & Risks** — an honest register of what it cannot do
- 10. **Roadmap** — the completed 7-day plan and beyond
- 11. **Why We Are Doing This** — strategic rationale
- 12. **Appendix** — metrics, commands, glossary

1 Executive Summary

NeuroTrace is Chrome DevTools for AI agents. It is a Python library that wraps an agent's execution loop, silently records a structured event for every decision the agent makes — each model call, each tool invocation, each error and retry, with timing and payloads — saves that recording to a local database, and renders it back as an inspectable timeline.

The business case rests on a single observation: **agent frameworks make it easy to build a loop and hard to see inside it.** When an agent misbehaves in production — loops on a tool, invents an argument, silently swallows a failure — the standard remedy today is to add `print()` statements and run it again, hoping the non-deterministic failure reproduces. NeuroTrace captures the run *once*, in full, so diagnosis becomes reading rather than re-rolling the dice.

~1,270 LINES OF LIBRARY CODE	58 TESTS, ALL PASSING	1 LINE OF CODE TO ADOPT	4+ LLM PROVIDERS SUPPORTED
--	---	---	--

What is complete

The product is finished end to end for v0.1.0 — run an agent, open a page, click through what it did. The event schema, the tracer, both storage backends (in-memory and SQLite), the OpenAI-compatible adapter with security validation, the command-line interface, a terminal timeline renderer, a read-only JSON API server, and the browser timeline viewer all work and are covered by tests. An agent adopts NeuroTrace by changing one construction line; the loop itself is untouched.

What is not

Streaming responses are not captured correctly — a streamed answer records only when it began, not its content. There is no redaction of sensitive data, and no async or threaded agent support. These are enumerated honestly in **Section 9**, because documentation that oversells a young library is worse than none — but none of them block the core loop of capturing a run and inspecting it.

THE ONE-SENTENCE PITCH

Instrument an agent in one line, and get a complete, causally-ordered, replayable record of everything it did — portable across OpenAI, xAI, Groq, and locally-hosted models, with no vendor SDK dependency and no data leaving the machine.

2 The Problem We Are Solving

2.1 What an agent is, and why it is hard to watch

An **AI agent** is a program that pursues a goal autonomously. Given an instruction — "find the cheapest flight to Chicago" — it does not execute a fixed script. It asks a language model what to do next, calls a tool (a function, an API, a database query), feeds the result back to the model, and repeats. The loop runs until the model declares itself done.

That loop is the product's value *and* its liability. The value: the agent handles situations nobody wrote code for. The liability: **nobody wrote code for those situations, so nobody knows what it did**. The developer sees an instruction go in and an answer come out. Everything between is invisible.

2.2 The four failure modes that cost real money

Failure	What it looks like	Business cost
Silent loop	The agent calls the same tool twenty times with slightly different arguments before giving up.	Direct token spend, multiplied. Latency the user experiences as a hang. Invisible until the invoice arrives.
Hallucinated call	The model asks for a tool that does not exist, or passes an argument the schema never offered.	A crash in production, or worse — a call that <i>succeeds</i> against a parameter you never intended to expose.
Swallowed error	A tool fails; the framework catches it, feeds a vague message back to the model, and the agent carries on to a wrong answer.	Wrong output delivered confidently. The most expensive class of failure, because nobody notices.
Non-reproducible bug	A user reports a bad run. Re-running the same prompt produces a different, working result.	Engineering hours burned on failures that cannot be reproduced on demand. The dominant cost in practice.

2.3 Why existing tooling does not close the gap

Three approaches exist today, and each falls short in a specific way:

- **Raw logging (`print` / `logging`).** Unstructured and ordered by wall-clock, so causality — *this* model call requested *that* tool — is lost. Reconstructing the loop from interleaved text is manual archaeology.
- **Framework-native tracing.** LangChain, LlamaIndex, and CrewAI each ship something, all mutually incompatible, all absent if you wrote the loop yourself against a raw SDK. Adopting one couples your observability to your framework choice.
- **Hosted platforms (LangSmith, Langfuse, Braintrust).** Capable, but they ship prompts and tool payloads to a third party — a non-starter for regulated data — and introduce a vendor dependency and a per-seat cost at exactly the stage where a team is still deciding whether agents work at all.

The gap NeuroTrace targets: **a framework-agnostic, locally-hosted, zero-dependency way to capture an agent run as structured, causally-linked data.**

ANALOGY THAT LANDS WITH NON-TECHNICAL STAKEHOLDERS

Before browser DevTools, debugging a slow web page meant guessing which request was slow. DevTools did not make pages faster — it made them *legible*, and the speed followed. NeuroTrace is that intervention, applied to agent loops.

3 The Product

3.1 Definition

NeuroTrace is an **observability layer for agent execution**. It is conceptually adjacent to distributed tracing systems (Jaeger, Honeycomb) — the same span/parent model — but purpose-built for reasoning loops rather than microservice request chains. Where a distributed trace answers "*which service was slow?*", a NeuroTrace answers "*why did the agent decide that?*"

3.2 The adoption experience

The design constraint that shaped the entire product: **instrumentation must not appear in the agent loop**. A developer wraps the client and changes nothing else.

```
# Before – an ordinary tool-calling loop
client = OpenAI()
response = client.chat.completions.create(model="gpt-4o", messages=messages)

# After – one line changes; the loop below is byte-identical
with Tracer(name="my-agent", storage=SQLiteStorage("traces.db")) as tracer:
    client = trace_openai(OpenAI(), tracer)
    response = client.chat.completions.create(model="gpt-4o", messages=messages)
    messages += client.dispatch_tool_calls(response, {"get_weather": get_weather})
```

The verified consequence: `examples/openai_agent.py` contains a complete agent loop with **zero NeuroTrace calls inside it**. Only the construction line knows tracing exists. This matters commercially — the adoption cost is near zero, and so is the cost of removing it if the team decides against it. Nothing is locked in.

3.3 The output

After a run, the trace is inspected from the terminal. The tree structure is the point: each tool call sits *under* the model call that requested it, so causality is visible at a glance rather than inferred from timestamps.

```
$ neurotrace view traces.db
Trace: weather-agent (d4561a4d) 2026-07-19T18:56:11 -> 2026-07-19T18:56:11
├─ llm_call gpt-4o 412.7ms
│   └─ tool_call get_weather 1.2ms
├─ llm_call gpt-4o 380.1ms
├─ llm_call gpt-4o 291.5ms
│   └─ tool_call book_flight 0.1ms
│       └─ error [error: no tool named 'book_flight']
└─ llm_call gpt-4o 244.8ms
```

Read that trace as a stakeholder: four model calls were billed, one tool succeeded, and the agent hallucinated a `book_flight` capability it was never given. That diagnosis took five seconds and no re-run.

3.4 Intended users

Audience	Job to be done
Agent developers	Diagnose a misbehaving loop without adding print statements and re-rolling a non-deterministic failure.
Teams evaluating providers	Run the identical loop against OpenAI, Groq, xAI, and a local model, and compare the traces on latency, token spend, and tool-calling reliability.
Security & compliance reviewers	Audit exactly which tools an agent invoked with which arguments — a record that exists nowhere else by default.
Regulated environments	Get hosted-platform-grade tracing without prompts or tool payloads leaving the machine.

4 Architecture

4.1 System layers

Four layers, deliberately built bottom-up: the schema everything speaks, the recorder, the translators for specific vendors, and the presentation surface.

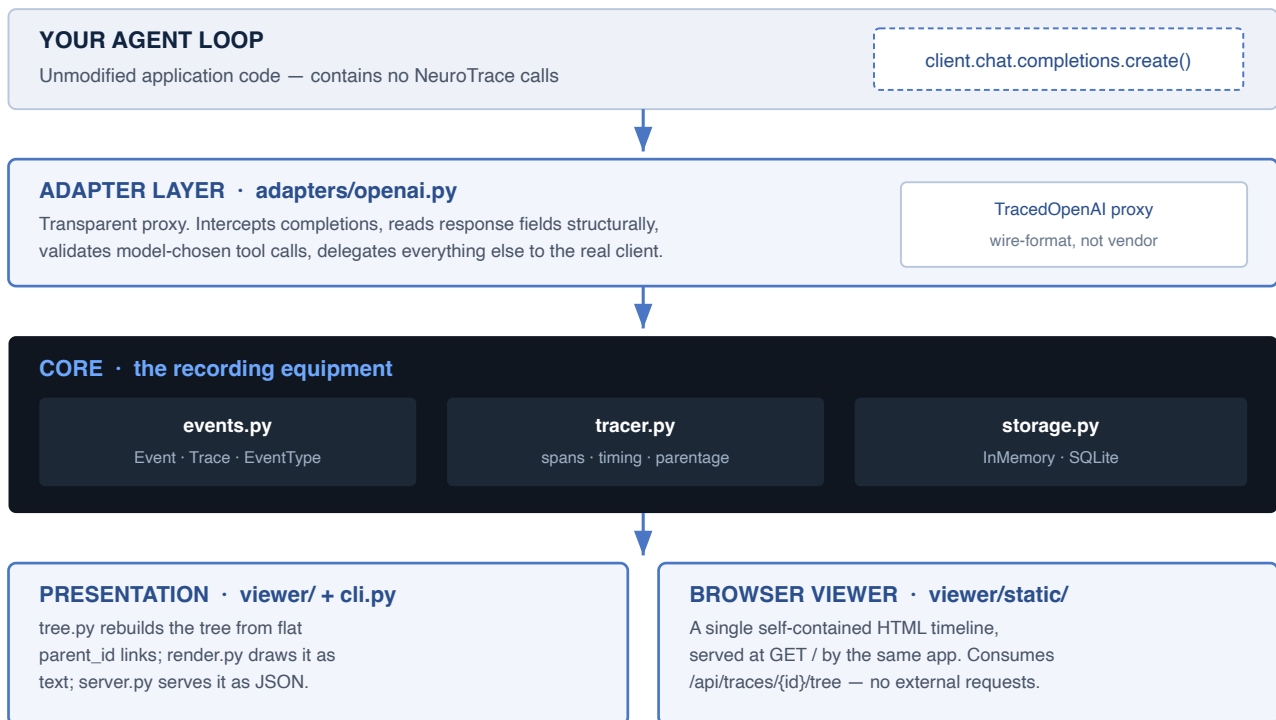


Figure 1 — System layers. Arrows indicate dependency direction: each layer knows only the one beneath it, so a second adapter (Anthropic, LangChain) plugs in without touching core, and the viewer works against any adapter's output.

4.2 Data flow

The pipeline from a live agent to a readable timeline. Note the single persistence point — a design decision with a consequence documented in Section 9.

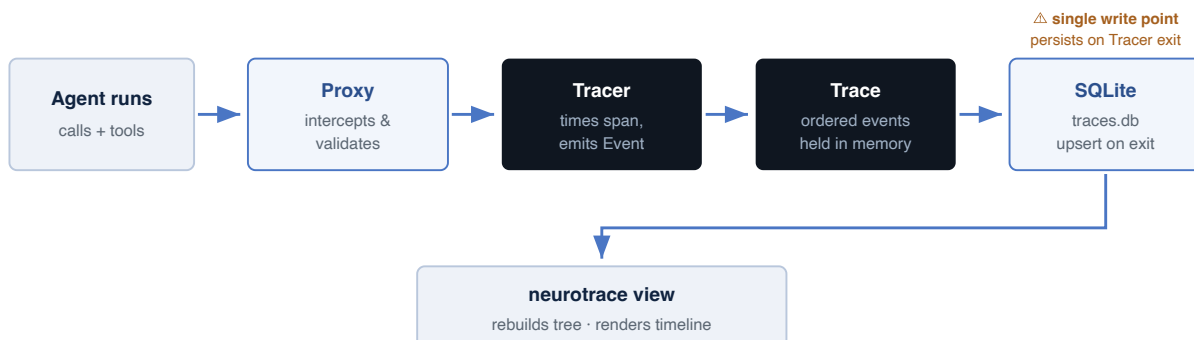


Figure 2 — Data flow. Events accumulate in memory during the run and are written once when the Tracer's context block exits. This keeps the hot path free of disk I/O, at the cost documented in Section 9.3.

4.3 Sequence of one traced agent turn

This is the most important diagram in the document, because it shows the ordering problem the architecture had to solve. The model *requests* a tool inside a completion, but the tool only *runs* after that completion has returned and its span has already closed.

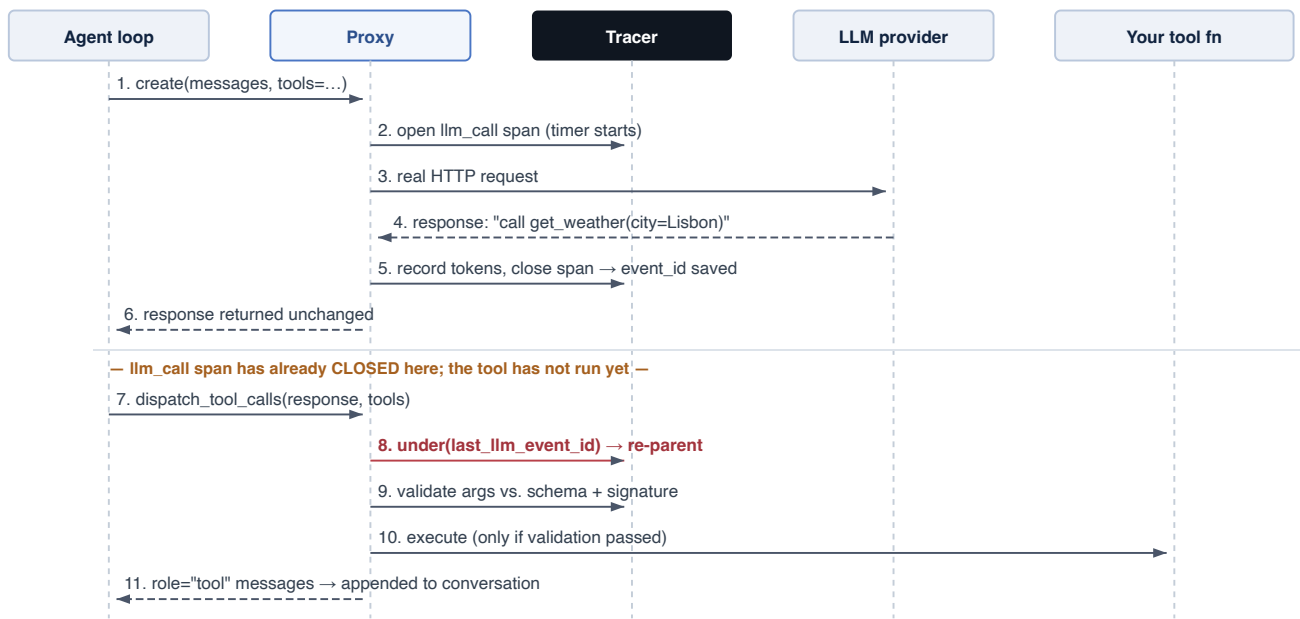


Figure 3 — One traced turn. Step 8 is the architectural crux: `Tracer.under()` re-parents the tool span under a completion span that has already closed, preserving the causal link the timeline depends on.

4.4 Flat storage, tree presentation

Events are stored as a flat list with a `parent_id` pointer. The tree is rebuilt at read time, not enforced at write time. This is why the event list's *order* differs from the timeline's *shape* — spans append when they close, so a parent appears in the list before its children even though it renders above them.

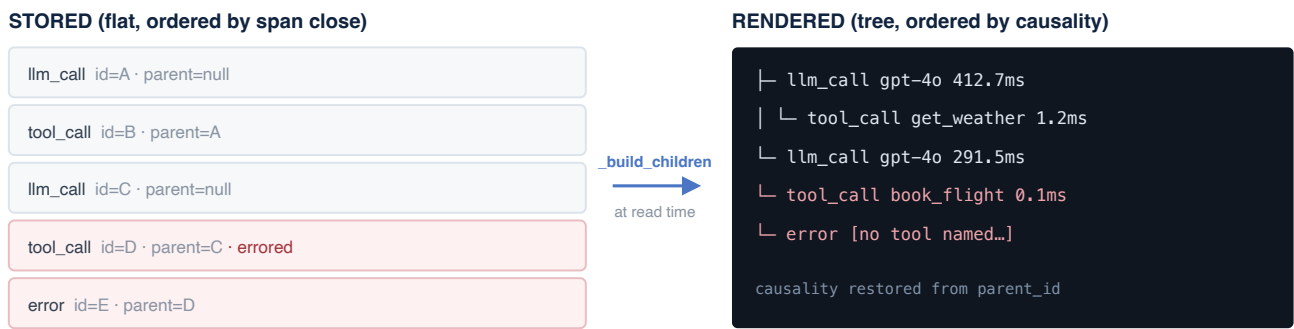


Figure 4 — Flat capture, tree presentation. Deferring tree construction to read time means capture never has to know the final shape, and any future viewer reuses the same `_build_children` logic rather than re-deriving it.

4.5 Data model

Two tables. A `Trace` is one agent run; an `Event` is one thing that happened inside it. The `payload` column is JSON, which is what makes the schema adapter-agnostic.

Table	Columns	Purpose & notes
traces	trace_id (PK), name, started_at, ended_at, metadata_json	One row per agent run. ended_at null means the run is still in progress or crashed. metadata_json carries free-form run context — the example stores which provider was used, enabling cross-provider comparison.
events	event_id (PK), trace_id (FK), event_type, payload_json, timestamp, duration_ms, parent_id, error	One row per recorded decision, indexed on trace_id. parent_id is a self-reference that encodes causality. Both tables use upsert, so re-saving an in-progress trace updates rows rather than duplicating them.

Event types

Type	Payload	Emitted by
llm_call	model, prompt, response, prompt_tokens, completion_tokens	Adapter, automatically on every completion
tool_call	tool_name, args, result	Adapter, automatically on every dispatched tool
error	message in the error column	Adapter on a rejected tool call; also manually via tracer.error()
retry	attempt, reason	Manual only — no automatic emitter yet
decision	—	Declared in the schema, never emitted — reserved for future reasoning-step capture

5 Component Walkthrough

Every file in the library, what it does, and why it earns its place.

`core/events.py` — the vocabulary (171 lines)

Defines `Event`, `Trace`, `EventType`, and four constructor functions. This was the first thing built, and deliberately so: every other component — recorder, storage, adapters, viewer — has to agree on what an "event" is, so the schema is the contract that makes parallel development possible.

Business value: a stable schema is what allows a second adapter (Anthropic, LangChain) or a second consumer (a dashboard, an export to OpenTelemetry) to be added later without renegotiating the data format. It is the asset with the longest half-life in the codebase.

`core/tracer.py` — the recorder (167 lines)

A context manager owning one `Trace` per run, with nested `llm_call` and `tool_call` spans that time themselves and infer parentage from a stack. Two details matter disproportionately:

- **Pre-generated event IDs.** A span's ID is created on *entry*, before the event object exists, so children can reference a parent that has not finished yet.
- **`__exit__` always persists** — including when the agent raised. A crashed run still leaves a partial trace, which is precisely the run you most want to inspect.

`core/storage.py` — persistence (165 lines)

One abstract interface, two implementations: `InMemoryStorage` for fast tests, `SQLiteStorage` for real runs. SQLite was chosen because it is in the Python standard library — a trace file is a single portable artifact with no server to run, no container to start, and no credentials to manage.

Business value: zero operational cost to adopt. A developer can attach a trace file to a ticket the way they would attach a log.

`adapters/openai.py` — the instrumentation (301 lines)

The largest and most commercially significant module. It is a transparent proxy: instrumented methods are traced, everything else is delegated via `__getattr__`. Three properties distinguish it:

- **The `openai` package is never imported.** Response fields are read structurally through a helper that handles both dictionaries and attribute objects. The consequence is Section 6.3.
- **Tool calls are validated before execution** — the security boundary described in Section 6.4.
- **Model mistakes and code failures are separated.** A hallucinated tool becomes a recorded error fed back to the model so the loop recovers; a tool that genuinely raises propagates unmodified so real bugs are not buried in a trace entry.

`viewer/tree.py` — the shared tree builder (53 lines)

Turns the flat, `parent_id`-linked event list back into a nested structure. Day 3 deliberately kept this logic out of the CLI on the grounds that a real UI would need it; Day 5's API server became that second consumer, so it moved here and both callers share one implementation rather than two that drift. It also re-attaches orphaned events as roots — an event pointing at a parent not present in the trace was previously dropped from the timeline entirely.

`viewer/render.py` — the terminal timeline (70 lines)

Presentation only, since the tree logic moved out. Returns a string rather than printing, so the CLI prints it, the API serves it, and tests assert on it directly.

`viewer/server.py` — the JSON API and viewer host (115 lines)

A read-only FastAPI application over one trace database. It exposes four JSON endpoints — trace summaries, one trace flat as stored, one trace nested as a timeline draws it, and one trace as rendered text — and serves the browser viewer itself at `GET /`. Three decisions carry weight:

- **A connection per request.** FastAPI runs synchronous endpoints in a worker threadpool, and SQLite rejects a connection used off its creating thread — so the tracer's single-connection design would fail intermittently here. This was verified as a real failure before being designed around, and the test issues repeated requests because a single one passes either way.
- **Read-only and loopback-bound by default.** There are no write endpoints, and the server binds to `127.0.0.1` rather than all interfaces, because traces hold prompts and tool arguments verbatim. Exposing them on a network is an explicit operator decision.
- **A missing database is an error, not an empty result.** Since connecting creates a schema, a typo'd path would otherwise serve a brand-new empty database and report zero traces — making a wrong path indistinguishable from an unused one.

`viewer/static/index.html` — the browser timeline (549 lines)

The visual product: a single self-contained page — vanilla JavaScript, inline styles, no build step and no framework — served at `GET /`. It lists runs on the left and, on selection, draws the chosen run's timeline as an expandable tree; each span shows a duration bar scaled to the slowest span, and clicks reveal the full prompt, response, or tool arguments and result. Two properties are enforced by tests: the page issues **no external requests** (a trace holds your prompts verbatim, so a viewer that fetched a CDN font would leak them), and it renders trace content through the DOM text API, **never as HTML** — the browser-side continuation of the adapter's stance that trace content is data to display, not code to run.

Business value: this is the piece a non-engineer can look at. It turns "we capture agent traces" from a claim into a two-minute demo.

`cli.py` — the interface (121 lines)

Three subcommands: `list`, `view`, and `serve` (the browser viewer plus the JSON API). `view` defaults to the most recently started trace, because the common need immediately after a run is "show me what just happened," not "let me look up an ID first." User errors (bad ID, empty database, missing file) exit with status 1 and a message on stderr rather than a traceback — they are expected conditions, not bugs.

`examples/openai_agent.py` — the proof (205 lines)

A complete tool-calling agent that runs **offline against a scripted client with no API key**, and switches to any of four real providers with a flag. The scripted conversation deliberately includes a hallucinated `book_flight` call, so the error path is visible in the very first trace a new user generates.

Business value: evaluation friction is near zero. A prospective user sees a real trace within a minute of cloning, without a credit card.

`tests/` — the safety net (58 tests, 833 lines)

Tests outnumber the modules they cover, with the adapter carrying the most (14) — appropriate, since it holds the security-sensitive logic. They run against hand-written fakes with no SDK installed, which is both faster and more honest: mocking the vendor SDK's types would only assert against our own assumptions about them.

6 Key Engineering Decisions

Five decisions shaped the product. Each is presented as the trade-off it actually was.

6.1 A flat event schema, not a class hierarchy

Decision: one `Event` type with a free-form `payload` dict, rather than `LLMCallEvent`, `ToolCallEvent`, and so on.

Why: the schema must survive two pressures a class hierarchy fights — round-trip serialization to SQLite and JSON, and adapters that describe the same concept differently (LangChain's callback payloads look nothing like raw OpenAI function-calling payloads). A flat schema means one table and one serializer.

Cost, stated plainly: weaker type safety. An LLM event and a tool event are both just "an Event with a dict." Constructor functions recover some of that at the call site without subclasses.

6.2 A proxy, not a monkeypatch

Decision: `trace_openai(client, tracer)` returns a wrapper, rather than globally patching the SDK's method.

Trade-off: a proxy cannot trace a client constructed somewhere you cannot reach. In exchange it is explicit about what is traced, nests correctly when two tracers are alive simultaneously, and does not break when the vendor reorganizes SDK internals — a real maintenance liability for any monkeypatching library.

6.3 Target the wire format, not the vendor

Decision: never import `openai`; read response fields structurally.

Initially a testing convenience, this turned out to be the project's most valuable decision. When cost pressure prompted a move off OpenAI, the migration required **zero adapter changes** — xAI, Groq, and Ollama all expose OpenAI-compatible endpoints, so swapping a `base_url` was the entire effort. Tracing, token capture, tool dispatch, and schema validation were verified unmodified against all three.

Provider	Base URL	Cost	Strategic role
OpenAI	<code>api.openai.com/v1</code>	Paid	Reference implementation
xAI (Grok)	<code>api.x.ai/v1</code>	Paid	Alternative frontier model
Groq	<code>api.groq.com/openai/v1</code>	Free tier	Zero-cost development and CI
Ollama	<code>localhost:11434/v1</code>	Free, local	Fully offline — nothing leaves the machine

COMMERCIAL READING

Provider portability converts a vendor lock-in risk into a procurement lever. The same agent code, and the same trace format, run against a frontier model in production and a free local model in development — and the traces are directly comparable, which makes "is the cheaper model good enough?" an empirical question rather than an argument.

6.4 The tool schema is a security boundary

This decision came out of a pre-publication review and closed a genuine vulnerability class. Dispatching a tool means calling a Python function with arguments *the model chose*. Two problems followed from splatting that JSON straight into a callable:

THE VULNERABILITY

A Python function may have parameters its published schema never offered the model:

```
def read_file(path, allow_absolute=False): # schema declares only `path`
```

Without a check, a prompt-injected tool call setting `allow_absolute=True` reaches that parameter and defeats a control the developer believed was private. The schema shown to the model is the contract; anything outside it must be unreachable.

The fix: arguments are restricted to the properties the schema declares, and then checked against the function's actual signature. Both come free — the schemas were already passed to `create(tools=...)`, so the proxy records them and applies them automatically.

A deliberate non-behaviour: an unknown schema means *skip the check*, not *deny everything*. Denying would break every caller who never passed one. And the check bounds argument *names*, not *values* — validating that a path stays inside an allowed directory remains the tool's own job. Overstating this control would be worse than not having it.

6.5 Model errors versus code errors

A consistent rule, applied at two levels:

Situation	Handling	Reasoning
Model names a tool that does not exist	Errored span, fed back to the model	This is agent behaviour NeuroTrace exists to reveal. It belongs in the trace, not in a traceback — and the loop can recover from it.
Model passes an argument the schema never offered	Errored span, fed back	
Model misspells or omits a required argument	Errored span, fed back	
Tool raises while executing	Propagates unmodified	This is ordinary application code failing. Swallowing it would hide a real bug behind a trace entry.
Tool arguments are malformed JSON	Kept verbatim under <code>_raw</code>	Unparseable arguments are the symptom; a decode error would destroy the evidence.

The split is now cleanly "*wrong call*" (recorded, recoverable) versus "*broken tool*" (raised). Before the Day 4a review, that line was drawn accidentally at whichever statement happened to throw first.

7 Capabilities

Verified against the current codebase and a passing test run.

Capability	Status	Detail
Automatic LLM call capture	Working	Model, flattened prompt, response text, prompt/completion tokens, wall-clock duration.
Automatic tool call capture	Working	Tool name, parsed arguments, return value, duration, error state.
Causal nesting	Working	Tools nest under the completion that requested them, across the span-closure boundary.
Tool-call validation	Working	Schema-declared parameters plus signature binding, applied before any invocation.
Crash-resilient persistence	Working	Partial traces are saved even when the agent raises.
Provider portability	Working	OpenAI, xAI, Groq, Ollama, and any OpenAI-compatible endpoint. Verified on all four.
Zero-dependency instrumentation	Working	No vendor SDK required; works against dicts, pydantic models, and recorded fixtures.
Terminal timeline + CLI	Working	<code>neurotrace list</code> , <code>view</code> , and <code>serve</code> , with sensible defaults.
Read-only JSON API	Working	Four endpoints over one trace database: summaries, flat trace, nested tree, rendered text. Loopback-bound by default.
Browser timeline viewer	Working	A single self-contained page served at <code>GET /</code> : run list, expandable tree, duration bars, payload detail. No external requests; trace content rendered as text.
Offline example agent	Working	Runs with no API key and produces a deterministic trace including an error path.
Retry capture	Manual	The API exists; no adapter emits it automatically, so SDK-internal retries are invisible.
Streaming responses	Not built	See Section 9.1 — currently records incorrect data rather than failing loudly.
Redaction / PII handling	Not built	See Section 9.2. The most significant gap for production adoption.
Async and concurrency	Not built	See Section 9.4.

8 Competitive Advantages

8.1 Near-zero adoption and exit cost

One construction line changes. The agent loop is untouched, which means the decision to adopt is cheap and — equally important to a cautious buyer — the decision to *remove* it is cheap too. There is no framework to migrate onto and no proprietary format to escape.

8.2 Data never leaves the machine

Traces are a local SQLite file. For teams handling regulated, confidential, or personal data, this is not a feature preference — it is the difference between usable and prohibited. Hosted competitors require shipping prompts and tool payloads to a third party.

8.3 Vendor-independent by construction

Because the adapter targets the wire format rather than a vendor SDK, provider migration costs a `base_url`. Traces from different providers share one format and are directly comparable — which turns model selection into a measurement rather than a debate.

8.4 Security posture as a differentiator

Most agent frameworks call `fn(**model_chosen_args)` without validation. NeuroTrace treats the published tool schema as an enforced boundary, and it does so by default because the schemas were already in the request. For a security reviewer, the trace also functions as an audit log: every tool invocation with every argument, including the ones that were rejected.

8.5 Extensibility already paid for

The flat schema, the adapter interface, and the read-time tree construction were designed for a second adapter and a second consumer before either existed. That the browser viewer consumes the same pre-nested tree the terminal renderer walks — rather than re-deriving it in JavaScript — is that bet paying off; adding Anthropic support is likewise a sibling module, not a rewrite.

8.6 A visible standard of engineering judgement

`docs/architecture.md` records not only what was decided but what was considered and rejected, and why. Two examples: a run-level error event was rejected because it double-counts what the span already recorded; renaming the module to `openai_compatible.py` was rejected because the accuracy gain does not justify a breaking import change. For a project intended to be public, this reasoning is itself a credential.

9 Limitations & Risks

READ THIS SECTION FIRST IF YOU ARE EVALUATING ADOPTION

NeuroTrace is a young v0.1.0 library. The capture-and-inspect loop is complete and solid; several properties expected of production observability infrastructure are still absent. They are listed here in descending order of consequence, not descending order of comfort.

9.1 Streaming responses record wrong data — silently

Calling `create(stream=True)` returns an iterator rather than a completed response. The adapter records an empty response body and a duration that measures only time-to-first-chunk. It does not error; it produces a trace that *looks* valid and is not.

Severity: high, because silent wrongness is worse than failure, and streaming is the default in most user-facing agents. **Fix:** wrap the iterator to accumulate deltas and close the span at the end — scheduled, and scoped as its own day of work.

9.2 Traces are unencrypted plaintext of everything the agent said

Prompts, responses, tool arguments, and tool results are written to SQLite verbatim. That is precisely what makes a trace useful — and it means a trace of a run whose system prompt embeds an API key, or whose tool arguments carry personal data, is now a plaintext copy of that secret on disk.

There is no redaction hook. Until there is, the operational controls are: keep `.db` files out of version control (already in `.gitignore`); treat a trace file as being as sensitive as the conversation it recorded, and read it before attaching it to a bug report; and place trace files where file permissions are appropriate when tracing production runs.

Severity: high for any regulated deployment. This is the single largest blocker between the current state and production use.

9.3 Persistence happens once, at the end of the run

Events accumulate in memory and are written when the `Tracer` block exits. Two consequences: a process killed hard (`SIGKILL`, OOM, power loss) loses the entire trace, including the crash you wanted to inspect; and a long-running agent holds every prompt and response in memory for its full duration. A run of thousands of steps with large payloads has an unbounded memory footprint.

Severity: medium. Incremental flushing is a natural extension — the storage layer already upserts, so re-saving an in-progress trace updates rather than duplicates.

9.4 Synchronous and single-threaded only

Three related gaps, one of which was closed on the server side in Day 5:

- **No async.** The proxy wraps a synchronous `create()`. An `AsyncOpenAI` client returns a coroutine the proxy neither awaits nor recognises — much production agent code is async.

- **Capture is not thread-safe.** The parent-tracking stack is an ordinary list with no thread-local isolation. Two agent threads sharing a tracer will interleave their stacks and produce a corrupted parent tree. *Unchanged.*
- **Connection not shareable** — **addressed for reads** The SQLite connection is created once in the constructor and Python's driver rejects cross-thread use. This was confirmed as a live failure and designed around in the API server, which opens a connection per request. The underlying `SQLiteStorage` object is still single-thread, so the constraint remains for any other concurrent consumer.

Severity: medium-high — capture still bounds the addressable use cases to single-threaded synchronous agents, which is a real fraction of the market but not the majority of production deployments.

9.5 The viewer is basic **shipped in Day 6**

The browser timeline now exists: a run list, an expandable tree, per-span duration bars, and payload detail on click, served at `GET /`. That closes the "Chrome DevTools for agents" positioning at the level of *inspecting one run*. What it does *not* yet do is the analysis layer that would make it a tool rather than a viewer: no text search across a trace, no filtering by event type or error, no side-by-side comparison of two runs, and no automatic flagging of pathologies like a repeated tool loop.

Severity: low for the core inspect-a-run use case, **medium** for the larger "analyser, not just viewer" ambition. The data plumbing for all of the above already exists in the API; these are additive UI features, not architectural gaps.

9.6 Prompt capture is lossy by design

The messages list is flattened into a single string for the event payload. This is adequate for reading a timeline and **insufficient for replaying a request** — the structure needed to reconstruct and re-run the exact call is discarded. If replay or regression-testing from traces becomes a goal, the payload format must change.

9.7 Coverage gaps in what is considered an "event"

- The `decision` event type is declared in the schema but never emitted anywhere. The reasoning-step capture the project's own framing promises is not implemented.
- `retry` events must be recorded manually. Retries performed inside the vendor SDK are invisible, so a trace can understate both latency and token spend.
- Errors raised *between* spans — in the agent's own orchestration code — are not recorded as events. The trace still saves, but with an unexplained gap.

9.8 Scale and performance are uncharacterised

- **Improved** Summary listing no longer loads every event of every trace. `list_trace_summaries()` is a single aggregate query with counts, used by both the CLI and the API's list endpoint. `list_traces()` retains the full-load behaviour, and the CLI's default "most recent trace" path still uses it to select a single row.
- Tracing overhead has never been measured. It is expected to be small relative to network latency, but "expected" is not "measured," and no benchmark exists.
- There is no sampling, retention policy, or size cap. A trace database grows without bound.

9.9 Ecosystem and interoperability

- **One protocol supported.** Only the OpenAI wire format. Anthropic's distinct token fields and content-block shape need a sibling adapter. LangChain, LlamaIndex, and CrewAI — named in the project's own framing — have no adapter at all, so the "framework-agnostic" claim is currently architectural rather than demonstrated.
- **No OpenTelemetry export.** Traces cannot flow into existing observability infrastructure, which for many organisations is a hard requirement rather than a nice-to-have.
- **Single-user, local-only.** No server, no authentication, no sharing model. A trace is shared by sending someone a file.
- **Tests use hand-written fakes.** A defensible choice, and it does mean no test exercises a real vendor SDK object. A silent upstream response-shape change would not be caught by CI.

9.10 Strategic risk

The clearest commercial risk is not technical. Agent observability is an actively contested category with funded entrants (LangSmith, Langfuse, Braintrust, Arize) and vendor-native tooling improving steadily. NeuroTrace's defensible positions are local-first data custody and true framework independence — both real, and both now demonstrable end to end in the shipped v0.1.0 rather than promised.

10 Roadmap

The 7-day plan, current position

Day	Status	Deliverable
1	Complete	Project skeleton and the Event schema — the vocabulary everything else speaks.
2	Complete	Tracer context manager, in-memory and SQLite storage.
3	Complete	CLI (<code>list</code> / <code>view</code>) and the text timeline renderer.
4	Complete	OpenAI adapter with auto-traced completions and nested tool calls.
4a	Complete	Tool-call validation — security follow-up from pre-publication review.
4b	Complete	Provider portability verified against xAI, Groq, and Ollama. Zero adapter changes.
5	Complete	Read-only JSON API server; tree-builder extracted for shared use; cheaper summary listing.
6	Complete	The browser timeline UI — expandable steps, duration bars, payload detail, highlighted errors. Consumes the Day 5 API; served at <code>GET /</code> .
7	Complete	Polish, documentation, Apache-2.0 license, changelog, packaging metadata — the v0.1.0 release.

Recommended priorities beyond v0.1.0

Ordered by the ratio of risk removed to effort required:

#	Item	Addresses	Rationale
1	Streaming support	§9.1	Currently produces silently wrong traces, and streaming is the default in user-facing agents. Wrong data is worse than no data.
2	Redaction hook	§9.2	The single largest blocker to production and regulated adoption. A payload-transform callback is modest work with disproportionate unlock.
3	Async client support	§9.4	Expands the addressable use cases substantially; most production agent code is async.
4	Incremental flush	§9.3	Makes hard crashes and long runs survivable. The upsert-based storage layer already supports it.
5	Anthropic adapter	§9.9	The first genuine test of the framework-agnostic claim — and the architecture was explicitly designed for it.
6	Cost estimation	New	Tokens are already captured. Multiplying by a price table turns a debugging tool into a budget tool, which changes who cares about it.
7	Loop detection	New	Automatically flagging "this agent repeated the same call five times" is the difference between a viewer and an analyser.
8	OpenTelemetry export	§9.9	Lets traces flow into existing infrastructure instead of competing with it.

11 Why We Are Doing This

11.1 The bet on where the difficulty moves

Building an agent is now easy; a working loop is an afternoon's work. **Trusting an agent remains hard.** As agents move from demos into paths where they spend money, touch customer data, and take actions with consequences, the binding constraint shifts from capability to verifiability. Tooling that answers "*what did it actually do, and why?*" becomes infrastructure rather than convenience. NeuroTrace is a position on that shift.

11.2 Why local-first, specifically

The hosted platforms are capable, and for many teams they are the right answer. But a meaningful segment cannot use them at any price, because their prompts and tool payloads cannot leave their infrastructure. That segment is currently served by `print()`. Serving it with a zero-dependency local library is a narrow, defensible position that does not require out-competing a funded platform on features.

11.3 Why framework-agnostic, specifically

Every agent framework ships its own tracing, and every one of them ties observability to a framework choice that teams are still actively revisiting. A trace format that survives switching frameworks — or writing the loop by hand against a raw SDK — is worth more over time than one that does not, because it is the artefact that accumulates history.

11.4 Why build it in this order

Core, then adapters, then viewer. A viewer with no real data to display is a mockup; an adapter with nowhere to send data is a stub. Building the recording equipment first meant every subsequent layer was validated against real captured traces rather than imagined ones. The visible cost was that the most demo-able component arrived last — a deliberate trade, and with the Day 6 viewer shipped, one now fully paid off.

11.5 What success looks like

- **Near term:** a developer diagnoses an agent failure from a trace instead of re-running with print statements, and it takes minutes rather than hours.
- **Medium term:** a team compares four providers on latency, token spend, and tool-calling reliability using identical code and directly comparable traces.
- **Long term:** the trace becomes the unit of record for agent behaviour — the thing attached to a bug report, reviewed in a security audit, and diffed between releases.

THE HONEST SUMMARY

NeuroTrace is a well-architected v0.1.0 that has solved the hard part — capturing causally-linked agent execution with near-zero adoption cost and genuine provider independence — and now ships the part that makes it obviously valuable to someone who has not read the code: a browser timeline they can click through. The engineering judgement on display is above the norm for a project this young. The remaining gaps — streaming, redaction, async — are real, documented honestly in Section 9, and none of them block the core loop of capturing a run and inspecting it.

12 Appendix

12.1 Project metrics

Metric	Value
Library source	~1,270 lines of Python across 9 modules, plus a 549-line self-contained browser viewer
Test suite	58 tests, 8 modules, all passing (verified 21 Jul 2026)
Example agent	205 lines, runs offline with no API key
Documentation	README (192 lines), architecture notes (311 lines), a changelog and Apache-2.0 license, and this document
Runtime dependencies	fastapi, uvicorn, pydantic — declared Day 1, first used Day 5
Vendor SDK dependencies	None. The <code>openai</code> package is never imported by the library.
Python requirement	≥ 3.10
Release	v0.1.0 — Apache-2.0 licensed, packaged as a wheel (viewer asset included)

Test distribution

Module	Tests	Focus
<code>test_openai_adapter.py</code>	14	Proxy delegation, tool dispatch, nesting, validation, malformed arguments
<code>test_server.py</code>	15	All four endpoints, 404s, read-only enforcement, threadpool survival, missing database, viewer serving, no-external-requests
<code>test_cli.py</code>	6	Subcommands, defaults, error exit codes
<code>test_storage.py</code>	6	Round-trip fidelity, upsert behaviour, both backends
<code>test_events.py</code>	5	Construction, serialization, ID uniqueness
<code>test_tracer.py</code>	5	Span timing, parent tracking, persistence on exception
<code>test_tree.py</code>	4	Nesting from <code>parent_id</code> , close-order independence, orphan recovery
<code>test_render.py</code>	3	Text output, error annotation, empty traces

12.2 Commands

```
# Install for development
pip install -e ".[dev]"
pytest

# Generate a trace offline – no API key required
python examples/openai_agent.py

# Or against a real provider
python examples/openai_agent.py --provider groq # openai | xai | groq | ollama

# Inspect
neurotrace list traces.db # every run, with event/error counts
neurotrace view traces.db # most recent run
neurotrace view traces.db --trace-id <id> # a specific run

# Serve the browser viewer + JSON API (loopback only by default)
neurotrace serve traces.db # open http://127.0.0.1:8756/ for the timeline
curl localhost:8756/api/traces # summaries
curl localhost:8756/api/traces/<id>/tree # nested timeline
```

12.3 Glossary

Term	Meaning
Agent	A program that uses a language model in a loop to accomplish a goal, calling tools along the way.
LLM call	One round trip to the model. Costs tokens and takes time — the dominant contributor to both cost and latency.
Tool call	The agent invoking an external capability — a function, an API, a database query — rather than the model itself.
Trace	The complete recorded history of one agent run, start to finish.
Event	One recorded occurrence within a run. A trace is an ordered list of events.
Span	An event with a duration — it opens, work happens, it closes. LLM calls and tool calls are spans.
Payload	The type-specific content an event carries; for an LLM call, the prompt, response, and token counts.
Adapter	Code translating a specific framework's or vendor's conventions into NeuroTrace's event format.
Tracer	The component that wraps the agent loop and does the recording — a black-box flight recorder.
Wire format	The shape of the request and response on the network, independent of any particular vendor's SDK. What the adapter actually targets.
Proxy	A wrapper object standing in for another, intercepting some calls and passing the rest through untouched.
Tool schema	The JSON description of a tool shown to the model — its name, purpose, and permitted parameters. In NeuroTrace, an enforced boundary.
Hallucinated call	The model requesting a tool that does not exist, or arguments that were never offered.
Upsert	Insert-or-update. Used so re-saving an in-progress trace refreshes rows rather than duplicating them.

12.4 Source documents

This document was compiled by reading the complete repository and revised at the v0.1.0 release after the Day 6 browser viewer and Day 7 packaging work: all nine library modules and the 549-line viewer, the example agent, all eight test modules, `README.md`, `docs/architecture.md`, `NeuroTrace-Explained.md`, `CHANGELOG.md`, `pyproject.toml`, and the git history. Capability claims were verified against the code; the 58-test result cited was produced by running the suite, and the API and viewer behaviour described in Sections 5, 7, and the walkthrough was verified against a live server over HTTP, on 21 July 2026.